Intelligent Slack Co-Pilot for Expert Matching

Musa Khan

A Slack-based intelligent system that converts unstructured user problems into structured queries and matches them with the most relevant experts using a hybrid reasoning engine.

Problem & Objective

The Challenge

- Hard to find the right person for specific technical problems
- Information scattered across unstructured conversations
- Expertise knowledge exists but isn't easily discoverable
- No centralized matching mechanism in Slack

System Objective

Build a Slack bot that understands user problems via conversation, structures the problem, matches experts, and returns ranked, explainable results.

System Architecture

Slack Bot (Bolt)

Real-time messaging interface using Bolt framework

Flask Server

HTTP entry point handling webhook requests

Request Handler

Routing layer directing conversation flow

Groq (LLM)

Language understanding and generation tasks

MongoDB

Knowledge base storing expert profiles

Hybrid System Design

🔵 Deterministic Layer

- Scoring engine
- Matching logic
- Ranking algorithms
- Field validation

🟣 Probabilistic Layer

- Question phrasing
- Information extraction
- Context understanding
- Response generation

> "I separated conversation control logic from natural language generation. The system decides what to ask; the LLM only phrases it."

This separation ensures **reproducible results** while maintaining **conversational flexibility**. The deterministic layer guarantees consistent matching behavior, while the probabilistic layer adapts to natural language variations.

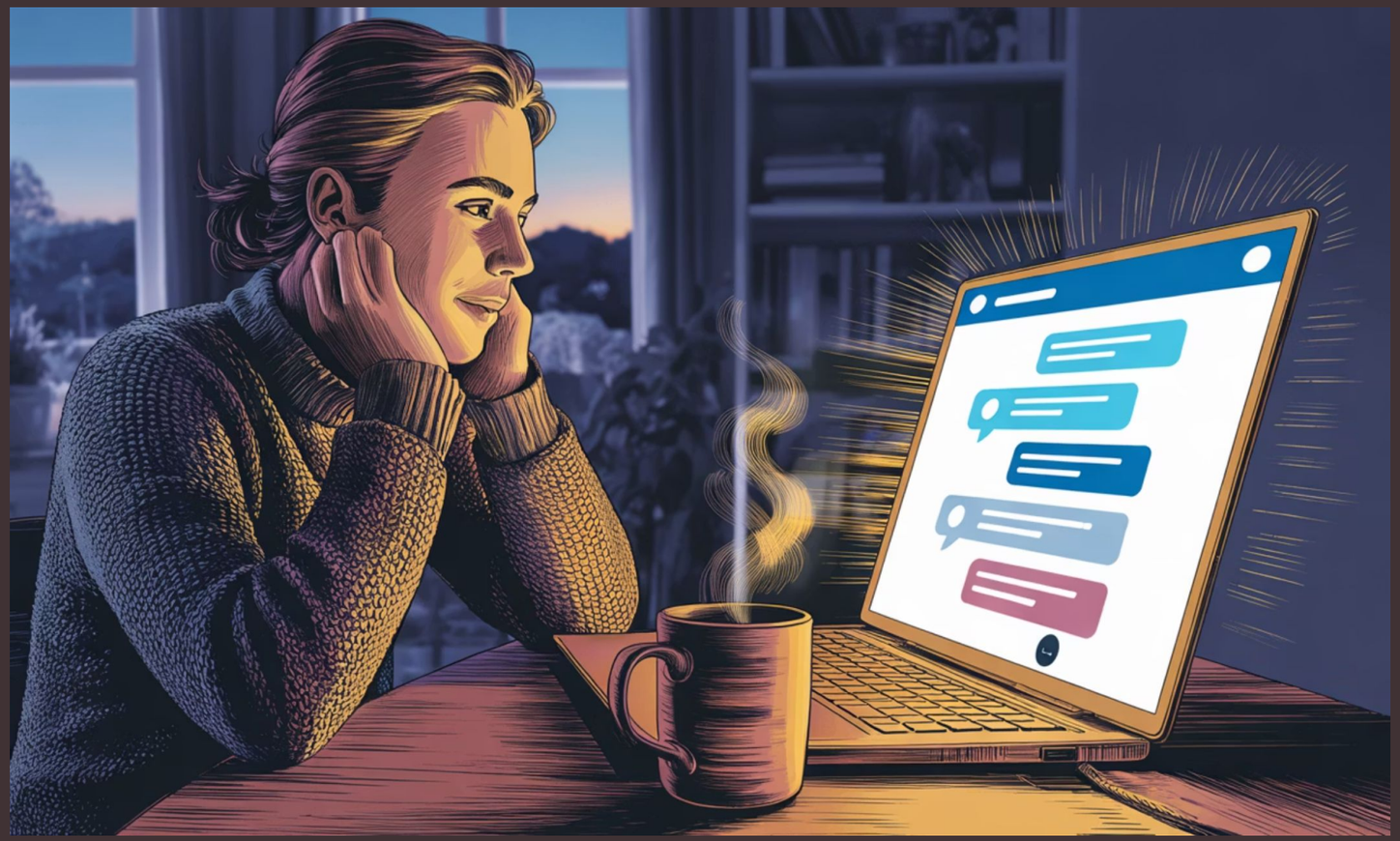
Conversation Engine

Multi-Turn Interaction

The bot engages users in a structured conversation, collecting essential information through a series of targeted questions.

Collected Information

- Core problem description
- Domain area
- Required tools
- Project context
- Prior attempts



State Management: In-memory session store keyed by user ID. Since Slack does not maintain conversational memory, I implemented custom state management to track conversation context across multiple interactions.

Intelligent Stopping & LLM Control

1

Minimum Questions

Continue if fewer than 7 questions asked

2

Maximum Questions

Stop if 10 or more questions reached

3

Field Completeness

Continue if required fields missing

4

Enrichment Check

Continue if low information gain

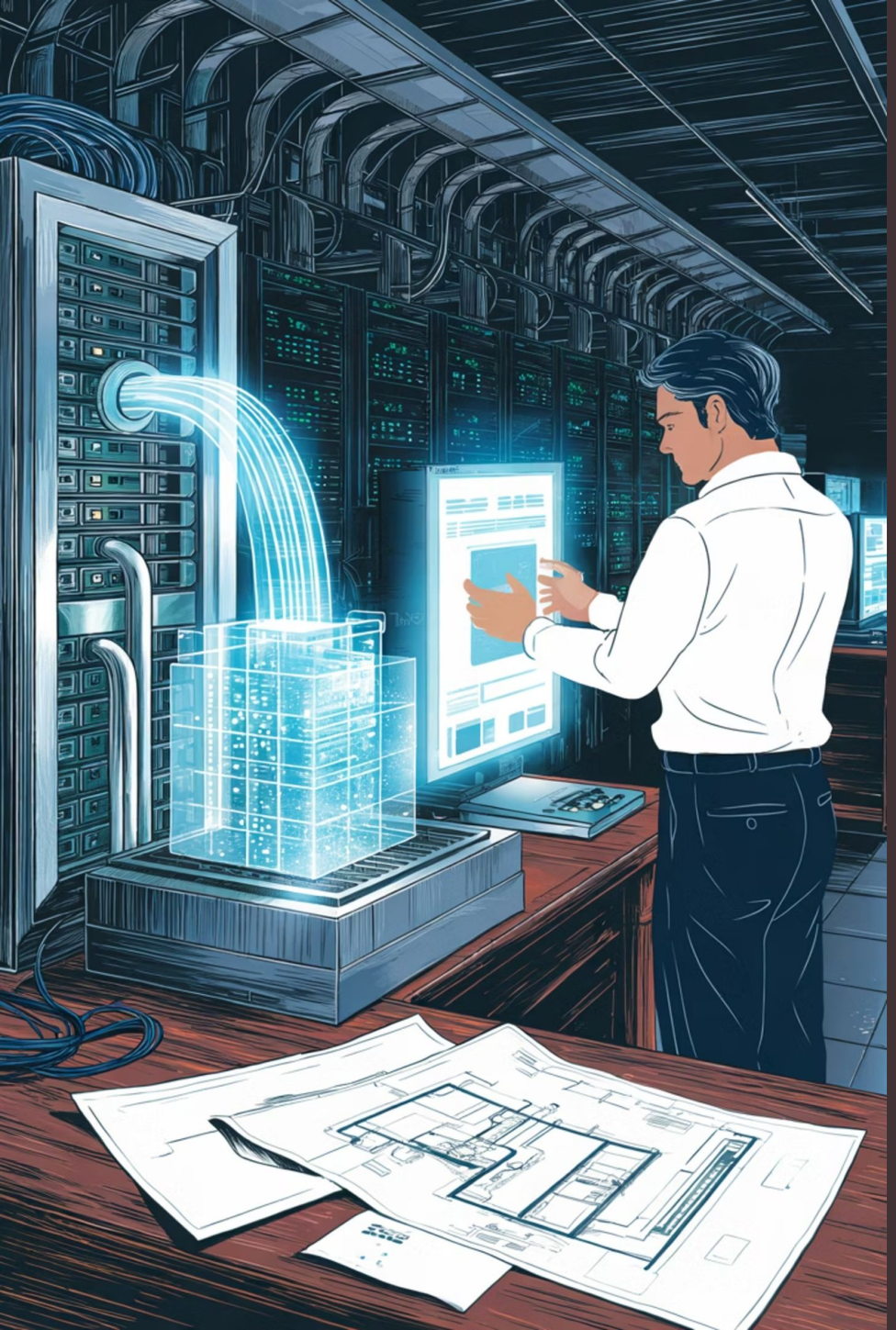
Temperature Configuration

Extraction: Temperature 0.0 (deterministic)

Reasoning: Temperature 0.3 (balanced)

Question phrasing: Temperature 0.45 (creative)

> "I designed a hybrid stopping criterion combining minimum conversation length and field completeness."



Normalization & Ontology Layer

01

User Input

Raw problem description in natural language

02

NEED_ALIASES

Normalize synonyms and variations

03

SKILL_GROUPS

Expand into structured skill clusters

04

Structured Space

Convert to queryable format

Key Insight: I introduced a skill ontology to convert abstract needs into structured skill clusters, enabling precise matching.

Normalization reduces **semantic drift** and ensures consistent matching across different ways users describe similar problems.

Matching Engine (Core)

Multi-Signal Scoring

Domain Match

Primary area alignment

Skill Overlap

Required vs. available skills

Tool Match

Specific technology fit

Experience

Years and depth

Project Relevance

Past project similarity

Two-Tier System

Direct matches: Strong alignment on all signals

Supporting matches: Partial relevance with explanation

> "I designed a multi-signal scoring engine to produce explainable and deterministic rankings."

Data Layer & Evaluation

MongoDB Schema (people collection)

```
{
  name: string,
  domain: string,
  skills: string[],
  tools: string[],
  experience: number,
  projects: string[]
}
```

Output Includes

- Ranked candidates
- Match scores
- Explainable reasoning

Why It Works

- Explainable logic
- Hybrid AI approach
- Structured reasoning

Limitations

- No semantic embeddings
- In-memory state only
- Small test dataset

Future Improvements

- Semantic embeddings
- Persistent sessions
- Real-world data

Live Demo

System in Action

Watch the conversation flow and expert matching process unfold in real-time. The bot guides users through structured questioning, normalizes inputs, and returns ranked expert matches with clear reasoning.

Start Demo

View Code